ACOB ODYSSEY

Metering Platform

Complete OpenAI Codex Build Guide

Step-by-step prompts to build the full-stack platform using OpenAI Codex — including real API endpoints from the Postman collection

Version	v2.0 — Management + Reports + Codex Prompts
Tool	OpenAI Codex (codex.openai.com) — agentic AI coding assistant
API Base URL	http://8.208.16.168:9310/api
Sites	KYAKALE · MUSHA · UMAISHA · TUNGA · OGUFA
Stack	React 18 + Vite + TypeScript + Tailwind + Node.js + Express
Auth	JWT HS256 Bearer token — stored in backend .env only
Date	10 March 2026

1. What Is OpenAI Codex?

OpenAI Codex is a cloud-hosted agentic software engineering assistant. Unlike traditional AI code completions, Codex operates inside a sandboxed Linux environment where it can read your codebase, write and edit files, run commands, run tests, and iterate until a task is complete — all from a single natural-language instruction.

For the ACOB Odyssey platform, Codex handles both the Node.js/Express backend and the React frontend. You give it a task description and it autonomously writes all the code, installs packages, and verifies the result.

1.1 Codex vs Lovable — When to Use Each

Use Case	Recommended Tool
Frontend UI only	Lovable is faster — visual preview + GitHub sync in seconds
Backend services & APIs	Codex — it can install npm packages, write TypeScript, run tests
Full-stack feature end-to-end	Codex — one prompt can touch backend service + API route + frontend component
Complex multi-file refactors	Codex — understands entire codebase context across all files
Non-developers	Lovable — simpler interface, no CLI required
Developers who use GitHub	Codex — integrates directly with GitHub repos and PRs

> 💡 **Best Practice**
> Use Lovable for rapid UI prototyping and Codex for all backend work and complex full-stack features. Both tools push code to the same GitHub repo.

1.2 How Codex Works

- You connect Codex to a GitHub repository (read/write access)
- You write a task in plain English — Codex reads all relevant files
- Codex runs in a sandboxed container: it writes code, installs packages, runs tests
- When finished, it opens a Pull Request (PR) to your repo for review
- You review the diff, approve, and merge — or ask Codex to revise

2. Architecture Overview

The platform is a two-tier monorepo. Codex works on the entire repo at once — it understands shared types in /common, the Express backend in /backend, and the React frontend in /frontend simultaneously.

```
┌──────────────────────────────────────────────────────────────┐
│              React Frontend  (/frontend)                     │
│  Vite · React 18 · TypeScript · Tailwind · Recharts          │
│                                                              │
│  /dashboard   /tokens   /management   /reports               │
└──────────────────────────────────────────────────────────────┘
                          │  Vite proxy → /api/*
┌──────────────────────────▼───────────────────────────────────┐
│           Node.js / Express Backend  (/backend)              │
│                                                              │
│  /api/dashboard   /api/tokens     /api/management            │
│  /api/reports     /api/auth       /api/operations            │
└──────────────────────────────────────────────────────────────┘
                          │  Bearer JWT  (ODYSSEY_JWT_TOKEN)
┌──────────────────────────▼───────────────────────────────────┐
│       ACOB Odyssey API   http://8.208.16.168:9310            │
│                                                              │
│  KYAKALE · MUSHA · UMAISHA · TUNGA · OGUFA                   │
└──────────────────────────────────────────────────────────────┘
```

2.1 Monorepo Structure

```
acob-odyssey/
├── common/
│   └── types/odyssey.ts          # Shared: SiteId, SITES[], TokenType
├── backend/
│   ├── .env                      # ODYSSEY_JWT_TOKEN (never commit!)
│   ├── src/
│   │   ├── config/index.ts       # Zod env validation + config object
│   │   ├── config/logger.ts      # Winston structured logger
│   │   ├── services/
│   │   │   ├── odyssey-client.ts    # Axios + retry + auth + pagination
│   │   │   ├── dashboard-service.ts # Promise.allSettled all 5 sites
│   │   │   ├── token-service.ts     # STS token generation
│   │   │   ├── management-service.ts# Gateway/Customer/Tariff/Account
│   │   │   └── report-service.ts    # 8 report types + CSV export
│   │   ├── api/
│   │   │   ├── dashboard.ts      # GET /api/dashboard (5-min cache)
│   │   │   ├── tokens.ts         # POST /api/tokens/*
│   │   │   ├── management.ts     # CRUD /api/management/*
│   │   │   └── reports.ts        # GET /api/reports/* + ?format=csv
│   │   └── middleware/
│   │       ├── auth.ts           # JWT verify middleware
│   │       └── error-handler.ts  # Global 404 + 500 handler
└── frontend/
    └── src/
        ├── pages/               # DashboardPage, TokensPage, etc.
        ├── components/          # Layout, Dashboard, Management, Reports
```

```
        ├── services/            # api.ts, management-api.ts, reports-api.ts
        └── hooks/               # useOdyssey.ts — all data hooks
```

3. Real API Endpoints (Postman Collection)

These are the confirmed, working endpoints from the ACOB_Odyssey_Api Postman collection. All requests require a Bearer JWT token in the Authorization header.

> 🔐 **Authentication**
> Every request must include: Authorization: Bearer <ODYSSEY_JWT_TOKEN> The JWT is HS256-signed and expires — check the exp claim. Rotate via the Odyssey admin panel when expired.

3.1 Credit Token Records

Retrieve the history of credit tokens generated for a site within a date range.

```
GET http://8.208.16.168:9310/api/token/creditTokenRecord/readMore

Query Parameters:
  FROM      2026-01-01T00:00:00.000Z   (ISO 8601 UTC — start of range)
  TO        2026-01-17T00:00:00.000Z   (ISO 8601 UTC — end of range)
  SITE_ID   KYAKALE                    (one of: KYAKALE|MUSHA|UMAISHA|TUNGA|OGUFA)

Headers:
  Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Response: paginated list of credit token records for the specified site.
  Each record contains: MeterSN, Amount, Token, GeneratedAt, OperatorId

⚠️  SITE PARTITIONING: A MUSHA meter will NOT appear in KYAKALE results.
   Query each site separately for cross-site aggregation.
```

3.2 Hourly Data (AMR)

Retrieve hourly electricity metering data (automatic meter readings) per site.

```
GET http://8.208.16.168:9310/api/DailyDataMeter/readHourly

Query Parameters:
  offset      0      (pagination offset)
  pageLimit   100    (records per page — max typically 1000)
  FROM        2026-01-10T00:00:00.000Z
  TO          2026-01-17T00:00:00.000Z
  SITE_ID     KYAKALE

Headers:
  Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Response: array of hourly meter readings.
  Each record contains: MeterSN, DateTime, ActiveEnergyImport,
```

```
  ActiveEnergyExport, ReactiveEnergyImport, Voltage, Current

Use this endpoint to power the Dashboard energy chart (hourly view).
```

3.3 Complete Endpoint Reference

All endpoints derived from the JWT scope claims in the Postman collection:

Endpoint	Description & Required Parameters
Token Generation	
POST /api/token/creditToken	Generate STS credit token. Body: { MeterSN, SITE_ID, Amount, Currency }
POST /api/token/clearTamperToken	Generate clear tamper token. Body: { MeterSN, SITE_ID }
POST /api/token/clearCreditToken	Generate clear credit token. Body: { MeterSN, SITE_ID }
POST /api/token/setMaximumPowerLimitToken	Set max power limit. Body: { MeterSN, SITE_ID, MaxPowerLimit }
POST /api/token/setMaximumOverdraftToken	Set max overdraft. Body: { MeterSN, SITE_ID, OverdraftLimit }
Token Records	
GET /api/token/creditTokenRecord/readMore	Credit token history. Params: FROM, TO, SITE_ID
GET /api/token/clearTamperTokenRecord/readMore	Clear tamper token history. Params: FROM, TO, SITE_ID
GET /api/token/clearCreditTokenRecord/readMore	Clear credit token history. Params: FROM, TO, SITE_ID
AMR Data	
GET /api/DailyDataMeter/readHourly	Hourly readings. Params: offset, pageLimit, FROM, TO, SITE_ID
GET /api/DailyData/read	Daily AMR data. Params: FROM, TO, SITE_ID
GET /api/MonthlyData/read	Monthly AMR data. Params: FROM, TO, SITE_ID
GET /api/DailyDataMeter/read	Daily per-meter data. Params: FROM, TO, SITE_ID, MeterSN
Remote Tasks	
POST /api/remoteMeterTask/createReadingTask	Create remote reading task. Body: { MeterSN, SITE_ID }

Endpoint	Description & Required Parameters
POST /api/remoteMeterTask/createControlTask	Create control task (connect/disconnect). Body: { MeterSN, SITE_ID, Action }
POST /api/remoteMeterTask/createTokenTask	Push token to meter remotely. Body: { MeterSN, SITE_ID, Token }
POST /api/remoteMeterTask/createSettingTask	Update meter settings. Body: { MeterSN, SITE_ID, Settings }
POST /api/gprsMeterTask/gprsCreateReadingTask	GPRS reading task. Body: { MeterSN, SITE_ID }
POST /api/gprsMeterTask/gprsCreateControlTask	GPRS control task. Body: { MeterSN, SITE_ID, Action }
Task Records	
GET /api/remoteMeterTaskRecord/getReadingTask	Get reading task status. Params: taskId, SITE_ID
GET /api/remoteMeterTaskRecord/getControlTask	Get control task status. Params: taskId, SITE_ID
GET /api/gprsMeterTaskRecord/gprsGetReadingTask	Get GPRS reading task. Params: taskId, SITE_ID
Prepay Reports	
GET /api/prepayReport/longNonpurchaseSituation	Meters inactive > N days. Params: SITE_ID, Days
GET /api/prepayReport/lowPurchaseSituation	Meters below spend threshold. Params: SITE_ID, Amount
GET /api/prepayReport/consumptionStatistics	Consumption stats. Params: FROM, TO, SITE_ID
Remote Reports	
GET /api/remoteReport/electricEnergyCurve	Energy curve (single phase). Params: MeterSN, FROM, TO, SITE_ID
GET /api/remoteReport/electricEnergyCurveThree	Energy curve (three phase). Params: MeterSN, FROM, TO, SITE_ID
GET /api/remoteReport/electricEnergyCurveCT	Energy curve (CT meter). Params: MeterSN, FROM, TO, SITE_ID

Endpoint	Description & Required Parameters
GET /api/remoteReport/instantaneousValueCurve	Instantaneous values. Params: MeterSN, SITE_ID
GET /api/remoteReport/dailyData	Daily report data. Params: FROM, TO, SITE_ID
GET /api/remoteReport/monthlyData	Monthly report data. Params: FROM, TO, SITE_ID
GET /api/remoteReport/eventNotification	Event log. Params: FROM, TO, SITE_ID, EventType (opt)
GET /api/remoteReport/gprsOnlineStatus	GPRS gateway status. Params: SITE_ID
Management	
GET /PUT /api/management/gateway	List / update gateways. Param: SITE_ID
GET /PUT /api/management/meter	List / update meters. Param: SITE_ID
GET /PUT /api/management/customer	List / update customers. Param: SITE_ID
GET /PUT /api/management/tariff	List / update tariffs. Param: SITE_ID
GET /PUT /api/management/account	List / update operator accounts
Settings	
GET /PUT /api/setting/station	Station configuration. Param: SITE_ID
GET /PUT /api/setting/role	Role definitions
GET /PUT /api/setting/user	User management
GET /api/setting/log	System audit log

4. JWT Scope Reference

The JWT token in the Postman collection contains all available permission scopes in its claims. These scopes gate every endpoint — a request will return 403 Forbidden if the token lacks the required scope.

> ⚠️ **Two Tokens in the Postman Collection**
> The collection contains two different tokens: CreditData uses a limited token (only TokenRecord.CreditTokenRecord + AutomaticMeterReading.DailyDataMeter scopes). DataHourly uses the full admin token with all scopes. Always use the full token in development.

4.1 Full Admin Token Scopes

The DataHourly request uses the full token. All available scopes grouped by module:

Module	Scopes
Token	Token.CreditToken · Token.ClearTamperToken · Token.ClearCreditToken · Token.SetMaximumPowerLimitToken · Token.SetMaximumOverdraftToken · Token.SetMaximumPhasePowerUnbalanceLimitToken · Token.MeterTestToken · Token.UpdateMeterKey · Token.ChangeMeterKeyToken
TokenRecord	TokenRecord.CreditTokenRecord · TokenRecord.CreditTokenCancelRecord · TokenRecord.CancelCreditTokenRecord · TokenRecord.ClearTamperTokenRecord · TokenRecord.ClearCreditTokenRecord · TokenRecord.SetMaximumPowerLimitTokenRecord · TokenRecord.SetMaximumOverdraftTokenRecord · TokenRecord.ChangeMeterKeyTokenRecord
RemoteMeterTask	RemoteMeterTask.CreateReadingTask · CreateSettingTask · CreateControlTask · CreateTokenTask · CreateTransparentForwardingTask · CreateUpdateFirmwareTask
GPRSMeterTask	GPRSMeterTask.GPRSCreateReadingTask · GPRSCreateSettingTask · GPRSCreateControlTask · GPRSCreateTokenTask
RemoteMeterTaskRecord	GetReadingTask · UpdateReadingTask · GetSettingTask · UpdateSettingTask · GetControlTask · UpdateControlTask · GetTokenTask · UpdateTokenTask
GPRSMeterTaskRecord	GPRSMeterTaskRecord.GPRSGetReadingTask · GPRSGetSettingTask · GPRSGetControlTask · GPRSGetTokenTask
PrepayReport	PrepayReport.LongNonpurchaseSituation · LowPurchaseSituation · ConsumptionStatistics
AutomaticMeterReading	AutomaticMeterReading.DailyData · DailyDataMeter · MonthlyData
RemoteReport	RemoteReport.ElectricEnergyCurve · ElectricEnergyCurveThree · ElectricEnergyCurveCT · InstantaneousValueCurve · InstantaneousValueCurveThree · InstantaneousValueCurveCT · DailyData

Module	Scopes
	· DailyDataThree · DailyDataCT · MonthlyData · MonthlyDataThree · MonthlyDataCT · EventNotification · GPRSOnlineStatus
Management	Management.Gateway · Management.Meter · Management.Customer · Management.Tariff · Management.Debt · Management.Account
Setting	Setting.Station · Setting.Role · Setting.User · Setting.Log · Setting.Item · Setting.Image

5. Setting Up OpenAI Codex

Step 1 **Access Codex**
Go to codex.openai.com

1. Navigate to https://codex.openai.com
2. Sign in with your OpenAI account (ChatGPT Plus or API subscription required)
3. You will see the Codex environment — a task input + file explorer panel

Step 2 **Connect Your GitHub Repository**
Link the ACOB Odyssey repo

4. Click 'Connect Repository' in Codex
5. Authorize Codex to access your GitHub account
6. Select the 'acob-odyssey' repository
7. Codex will clone the repo into its sandbox — this takes ~30 seconds

> 💡 **First Time Setup**
> If you haven't created the repo yet, create an empty one on GitHub first, then push the project files. Codex needs an existing repo to connect to.

Step 3 **Set Environment Variables in Codex**
Provide the JWT token securely

8. In the Codex environment, click 'Secrets' or 'Environment Variables'
9. Add: ODYSSEY_JWT_TOKEN = <your full JWT from the Postman collection>
10. Add: ODYSSEY_API_URL = http://8.208.16.168:9310/api
11. These are injected into the sandbox at runtime — never stored in code

> 🔐 **Security Rule**
> Never paste the JWT token directly into a Codex prompt. Always use Secrets. The token is a long-lived credential — if leaked, rotate it immediately via the Odyssey admin panel.

6. Building the Backend with Codex

Each section below contains a Codex task prompt. Paste it into the Codex task input and let it run. Codex will write code, install packages, and run TypeScript compilation checks.

6.1 Task: Odyssey HTTP Client

This is the foundational service — build it first.

Step 4 **Build the Core Odyssey HTTP Client**
backend/src/services/odyssey-client.ts

```
Task for Codex:

Create backend/src/services/odyssey-client.ts — an Axios-based HTTP client
for the ACOB Odyssey API at http://8.208.16.168:9310/api.

Requirements:
- Read ODYSSEY_JWT_TOKEN and ODYSSEY_API_URL from process.env
- Set Authorization: Bearer {token} on every request automatically
- Implement exponential backoff retry: MAX_RETRIES=3, BASE_DELAY_MS=1000
- Skip retry on 4xx status codes (except 429 Too Many Requests)
- Implement fetchAllPages() that automatically paginates using offset/pageLimit
  until all records are retrieved
- Export a typed get<T>(), post<T>(), and fetchAllPages<T>() method
- Export fetchAllSites() — runs a fetcher against all 5 sites in parallel
  using Promise.allSettled so a single site failure doesn't break the rest
- Sites constant: ['KYAKALE', 'MUSHA', 'UMAISHA', 'TUNGA', 'OGUFA']
- Log all requests and retries using the Winston logger from config/logger.ts

Key endpoints to support (from Postman collection):
  GET /token/creditTokenRecord/readMore?FROM=&TO=&SITE_ID=
  GET /DailyDataMeter/readHourly?offset=&pageLimit=&FROM=&TO=&SITE_ID=
```

6.2 Task: Dashboard Service

Step 5 **Build the Dashboard Service**
KPI aggregation across all 5 sites

```
Task for Codex:

Create backend/src/services/dashboard-service.ts.

getDashboardKPIs(): fetch data from all 5 sites in parallel, return:
  - totalActiveMeters: count of meters with readings in last 24h
  - totalRevenue: sum of all credit token amounts (NGN)
```

```
  - totalKwhToday: sum of ActiveEnergyImport from hourly data
  - gprsOnlinePercent: % of gateways with ONLINE status
  - perSite: array of { siteId, meters, revenue, kwh, gprsStatus }

getHourlyData(siteId, from, to): calls GET /DailyDataMeter/readHourly
  Params: offset=0, pageLimit=100, FROM={from}, TO={to}, SITE_ID={siteId}
  Returns array of { hour, kwh, voltage, current }

getGprsStatus(): calls GET /remoteReport/gprsOnlineStatus for all 5 sites

getEventNotifications(from, to, siteId?): calls GET
/remoteReport/eventNotification

Cache getDashboardKPIs() for 5 minutes using a simple in-memory Map.
Log timing for each multi-site fetch.
```

6.3 Task: Token Service

Step 6 **Build the Token Service**

STS token generation + records

```
Task for Codex:

Create backend/src/services/token-service.ts.

generateCreditToken(meterSN, siteId, amount, currency='NGN'):
  POST /api/token/creditToken
  Body: { MeterSN, SITE_ID, Amount, Currency }
  Return: { token: string, meterSN, amount, generatedAt }

generateClearTamperToken(meterSN, siteId):
  POST /api/token/clearTamperToken

generateClearCreditToken(meterSN, siteId):
  POST /api/token/clearCreditToken

generateMaxPowerToken(meterSN, siteId, maxPowerLimit):
  POST /api/token/setMaximumPowerLimitToken

getCreditTokenRecords(siteId, from, to, offset=0, pageLimit=20):
  GET /api/token/creditTokenRecord/readMore
  Query: FROM, TO, SITE_ID, offset, pageLimit
  This is the confirmed working endpoint from the Postman collection.

All methods must validate siteId is one of the 5 valid sites.
Log operator, meterSN, and amount for every token generated.
```

6.4 Task: Register All API Routes

```
Task for Codex:

Update backend/src/index.ts to register all routes:

app.use('/api/dashboard', dashboardRouter)    // 5-min cache KPIs
app.use('/api/tokens',    tokensRouter)       // token generation + records
app.use('/api/management',managementRouter)   // CRUD 4 entities
app.use('/api/reports',   reportsRouter)      // 8 report types + CSV

Also add:
- Rate limiting: 100 req/15min using express-rate-limit
- Helmet security headers
- CORS: allow origins from process.env.CORS_ORIGINS (comma-separated)
- Morgan request logging in dev, combined format in production
- Global error handler: return { success: false, error: message, code: status }
- 404 handler for unmatched routes
- GET /health endpoint: return { status: 'ok', uptime, env, timestamp }

After updating, run: npx tsc --noEmit
Fix any TypeScript errors before finishing.
```

7. Building the Frontend with Codex

7.1 Task: Dashboard Page

Step 7 **Build the Dashboard Page**
KPI cards + hourly chart + site table

```
Task for Codex:

Build frontend/src/pages/DashboardPage.tsx.

Layout (top to bottom):
  1. Row of 4 KPI StatCards:
     - Total Active Meters   (icon: Activity,    accent: #00B4D8)
     - Total Revenue ₦       (icon: DollarSign,  accent: #06D6A0)
     - Total kWh Today       (icon: Zap,         accent: #FFB703)
     - GPRS Online %         (icon: Wifi,        accent: #FB8500)

  2. Two-column row:
     Left (60%): EnergyChart — Recharts AreaChart of hourly kWh per site.
       Site colors: KYAKALE #06D6A0, MUSHA #00B4D8, UMAISHA #FFB703,
       TUNGA #FB8500, OGUFA #EF4444. Data from GET /api/dashboard/hourly.
     Right (40%): EventsFeed — scrollable list of recent meter events.

  3. SiteBreakdown table — columns: Site | Meters | Revenue | kWh | GPRS
     Color each site name with its corresponding color.

Data: useQuery from GET /api/dashboard, auto-refetch every 5 minutes.
Show animated shimmer skeletons while loading.
Dark theme: bg-[#0A1628], cards bg-[#0F1E35], borders rgba(30,48,80,0.8).
```

7.2 Task: Token Management Page

```
Task for Codex:

Build frontend/src/pages/TokensPage.tsx.

Top section — Token Generator form:
  Fields: Site (select), Meter SN (text), Amount NGN (number ≥100),
  Token Type (Credit / Clear Tamper / Clear Credit / Max Power)
  Submit → POST /api/tokens/credit (or relevant endpoint by type)
  On success: show the 20-digit STS token as XXXXX-XXXXX-XXXXX-XXXXX
  in a large display box with Copy to Clipboard button (cyan glow).
  Token display box background: #0D1B2E, border: 2px solid #00B4D8.

Bottom section — Token Records table:
  Columns: Site | Meter SN | Amount | Token (masked: ●●●●●-●●●●●-XXXXX-XXXXX) |
```

```
            Type | Generated At
  Clicking the masked token reveals it (toggle show/hide).
  Filters: Site dropdown, date range picker (FROM/TO), search by Meter SN.
  Data from GET /api/tokens/records?siteId=&from=&to=&offset=&pageLimit=20
  Pagination: previous/next buttons, showing 'Page X of Y'.
  Export CSV button: triggers download from /api/tokens/records?format=csv
```

7.3 Task: Management Module

```
Task for Codex:

Build frontend/src/pages/ManagementPage.tsx with 4 tabs:

TAB: Gateways
  Card grid (3 cols). Each card: name, site badge, IP, protocol,
  location, ONLINE/OFFLINE status (green wifi / gray wifi-off icon).
  Add Gateway button → modal with: Name, Site, IP Address,
  Protocol (GPRS/Ethernet/WiFi/RS485/LoRa), Location, Notes.
  Edit pencil icon + red Trash icon on each card.
  Decommission requires a confirmation modal.

TAB: Customers
  Searchable table: Customer | Account No | Meter SN | Site | Tariff | Status
  Status badge: ACTIVE=green, INACTIVE=gray, SUSPENDED=amber.
  Site filter dropdown + search input (searches name and meter SN).
  Add Customer modal: Name, Account Number, Meter SN, Site, Tariff ID,
  Phone, Status dropdown, Address.

TAB: Tariffs
  Card grid. Each card: name, site badge, large ₦X.XX rate display,
  Effective From/To dates, approver ID, Active/Expired badge.
  Amber warning banner: 'Rate changes require supervisor approval.'
  Create Tariff modal: Name, Site, Rate/kWh, Currency, Dates, Approved By.

TAB: Accounts
  Card grid. Each card: initials avatar, operator name, username,
  role ID, scope count, active status icon.
  Create Account: username, name, role, email, scope picker (grouped).
  Deactivate requires entering revoker ID in confirmation modal.

API: /api/management/gateways, /customers, /tariffs, /accounts
```

7.4 Task: Data Reports Page

```
Task for Codex:

Build frontend/src/pages/ReportsPage.tsx.

Horizontal scrollable tab bar with 8 tabs:
```

```
  Non-Purchase | Low Purchase | Consumption Stats | Daily Yield |
  Monthly Yield | Daily AMR | Monthly AMR | Event Log

Filter bar (adapts to active tab):
  - Site dropdown (All + 5 individual sites)
  - Date range: FROM/TO date inputs
  - Quick presets: 7d / 30d / 90d / 1y buttons
  - Non-Purchase only: 'Inactive >= N days' number input
  - Low Purchase only: 'Total ≤ ₦X' number input
  - Event Log only: Event Type dropdown

KPI strip: 5 small cards showing record count per site (colored dots).

Bar chart (Recharts BarChart): record count per site,
  each bar colored with the site's color.

Data table: Site column + report-specific columns (see below).
Export CSV: anchor href to /api/reports/{type}?format=csv&{params}

Column definitions:
  non-purchase:  Meter SN | Last Purchase | Days Inactive | Customer
  low-purchase:  Meter SN | Total Amount  | Purchases     | Customer
  consumption:   Meter SN | Total kWh     | Avg Daily     | Peak kW
  daily-yield:   Date     | Energy kWh    | Meters        | Revenue
  daily-amr:     Meter SN | Date          | Import kWh    | Export kWh | Peak kW
  events:        Meter SN | Event (amber) | Timestamp     | Description
```

8. Remaining Modules — Codex Prompts

8.1 Authentication / Login

Priority: HIGH — build before deploying to any environment.

```
Task for Codex:

Add a complete authentication flow:

BACKEND — backend/src/api/auth.ts:
  POST /api/auth/login
    Body: { username: string, password: string }
    Validates credentials against Setting.User records via Odyssey API
    On success: returns { token: string, expiresIn: number, role: string }
    On failure: returns 401 { success: false, error: 'Invalid credentials' }

  POST /api/auth/refresh
    Body: { token: string }
    Returns new token if existing token is valid and within 24h of expiry

FRONTEND:
  Create frontend/src/pages/LoginPage.tsx:
    Centered card on dark background (#0A1628).
    ACOB ODYSSEY logo + subtitle 'Metering Operations Platform'.
    Username input + Password input (show/hide toggle).
    'Sign In' button — full width, background #10A37F (green).
    Error message area (red callout) for failed logins.
    POST to /api/auth/login — store JWT in localStorage as 'odyssey_token'.

  Create frontend/src/components/ProtectedRoute.tsx:
    Check localStorage for 'odyssey_token'.
    Decode JWT to check exp — redirect to /login if expired.
    Otherwise render children (outlet).

  Update App.tsx: wrap all routes except /login in <ProtectedRoute>.
  Update sidebar: add logout button at the bottom.
    Clicking logout: clears 'odyssey_token' and redirects to /login.

  Update all API service files to include:
    Authorization: Bearer {localStorage.getItem('odyssey_token')}
    on every request.
```

8.2 Remote Operations

```
Task for Codex:

Build the Remote Operations module.
```

```
BACKEND — backend/src/services/remote-task-service.ts:
  createReadingTask(meterSN, siteId):
    POST /api/remoteMeterTask/createReadingTask
    Body: { MeterSN, SITE_ID }
    Returns: { taskId, status: 'PENDING' }

  createControlTask(meterSN, siteId, action: 'CONNECT'|'DISCONNECT'):
    POST /api/remoteMeterTask/createControlTask
    DISCONNECT requires dual-auth — enforce at route level (see below)

  pollTaskStatus(taskId, siteId, type: 'reading'|'control'):
    GET /api/remoteMeterTaskRecord/getReadingTask?taskId=&SITE_ID=
    Returns: { status: 'PENDING'|'RUNNING'|'SUCCESS'|'FAILED', result? }

  createGprsReadingTask(meterSN, siteId):
    POST /api/gprsMeterTask/gprsCreateReadingTask

BACKEND — backend/src/api/operations.ts:
  POST /api/operations/task — create any task type
  GET  /api/operations/task/:id — poll task status
  POST /api/operations/disconnect — REQUIRES both operatorId AND supervisorCode
    Return 403 if supervisorCode is missing or invalid

FRONTEND — frontend/src/pages/OperationsPage.tsx:
  Task Launcher: Task Type dropdown, Site, Meter SN, Action (for control tasks)
  DISCONNECT: shows DisconnectModal requiring Operator ID + Supervisor Code
  Task Status Tracker: polls GET /api/operations/task/:id every 30 seconds
    Max 10 polls (5-minute timeout). Shows: PENDING → RUNNING → SUCCESS/FAILED
    Status badges: PENDING gray, RUNNING amber pulse, SUCCESS green, FAILED red
  GPRS Status Grid: 5-site grid showing gateway online/offline + last seen time
    Data from GET /api/remoteReport/gprsOnlineStatus for each site
```

8.3 Settings & Admin

```
Task for Codex:

Build frontend/src/pages/SettingsPage.tsx with 3 tabs:

TAB: Roles
  List all roles from GET /api/setting/role
  Each role card: role name, ID, list of assigned permissions.
  Create Role button → modal with name + scope selector.

TAB: Station Config
  Site selector dropdown (5 sites).
  On selection, fetch GET /api/setting/station?SITE_ID=
  Editable fields: Max Power Limit (kW), Low Credit Threshold (₦),
  Non-Purchase Alert Days, Meter Communication Interval (min).
  Save button → PUT /api/setting/station
  Show amber warning: 'Station changes affect live meters. Verify with site
engineer.'

TAB: System
```

```
  App version, API connection status (green/red dot, test with GET /health),
  Current cache TTL (editable), Log level selector (info/debug/warn),
  Audit log: last 50 entries from GET /api/setting/log
```

8.4 Docker Production Deployment

```
Task for Codex:

Add Docker production deployment configuration.

Create backend/Dockerfile:
  FROM node:20-alpine AS builder
  WORKDIR /app
  COPY backend/package*.json ./
  RUN npm ci
  COPY backend/src ./src
  COPY common ./common
  COPY backend/tsconfig.json .
  RUN npx tsc

  FROM node:20-alpine
  WORKDIR /app
  COPY --from=builder /app/dist ./dist
  COPY --from=builder /app/node_modules ./node_modules
  EXPOSE 4000
  CMD ["node", "dist/index.js"]

Create frontend/Dockerfile:
  FROM node:20-alpine AS builder
  WORKDIR /app
  COPY frontend/package*.json ./
  RUN npm ci
  COPY frontend ./
  COPY common ./common
  ARG VITE_API_URL
  ENV VITE_API_URL=$VITE_API_URL
  RUN npm run build

  FROM nginx:alpine
  COPY --from=builder /app/dist /usr/share/nginx/html
  COPY frontend/nginx.conf /etc/nginx/conf.d/default.conf
  EXPOSE 80

Create docker-compose.prod.yml with backend + frontend services.
Create frontend/nginx.conf with SPA routing (try_files for React Router).
Add a .github/workflows/deploy.yml GitHub Actions pipeline:
  On push to main: build both Docker images, push to GitHub Container Registry.
```

9. Codex Prompting Best Practices

9.1 Writing Effective Task Prompts

Practice	Why It Works
Name the exact file	Start with 'Create backend/src/services/report-service.ts' — Codex knows exactly which file to write or edit
State the HTTP contract	Include the full endpoint URL, method, query params, and expected response shape. Codex won't guess.
Reference the Postman data	Say 'the confirmed working endpoint is GET /api/token/creditTokenRecord/readMore (from Postman collection)'
Specify TypeScript strictness	Say 'no any types — use the shared SiteId type from common/types/odyssey.ts'
Request validation	Say 'validate with Zod and return 400 with error details if validation fails'
Ask for tests	Add 'write a Jest unit test in backend/src/__tests__/' — Codex will write and run the tests
Mention related files	Say 'follow the pattern in dashboard-service.ts' — Codex reads both files for consistency
Set scope boundaries	Say 'do not modify index.ts or any existing routes' to prevent unintended changes

9.2 Iterating on Codex Output

- Review the PR diff carefully — Codex sometimes adds more than requested
- If a file was modified unexpectedly, comment on the PR: 'Please revert changes to X.ts and keep the original'
- For bugs: paste the error message + stack trace as a new task
- For styling: paste a screenshot description ('the card text is white-on-white — fix contrast')
- Codex remembers the previous task in a session — use 'also add...' for incremental changes

9.3 Quality Checks After Each Task

After Codex completes a task, always verify:

```
# 1. TypeScript — no errors
cd backend && npx tsc --noEmit
cd frontend && npx tsc --noEmit

# 2. Run and test the backend health endpoint
npm run dev
curl http://localhost:4000/health
```

```
# 3. Test a real API call (using confirmed Postman endpoint)
curl 'http://localhost:4000/api/tokens/records?siteId=KYAKALE&from=2026-01-
01T00:00:00.000Z&to=2026-01-17T00:00:00.000Z'

# 4. Check for any console errors in the frontend
cd frontend && npm run dev
# Open http://localhost:5173 and check browser console
```

10. Troubleshooting

Problem	Solution
401 from Odyssey API	JWT expired. Decode it at jwt.io to check exp claim. Generate a new token via the Odyssey admin panel and update ODYSSEY_JWT_TOKEN in Codex Secrets and local .env
CreditData token returns 403	The CreditData Postman token only has 2 scopes (TokenRecord.CreditTokenRecord + DailyDataMeter). Use the full DataHourly token for all other endpoints.
MUSHA/UMAISHA returns empty data	Meters are site-partitioned. A meter registered on MUSHA will NOT appear in KYAKALE results. Confirm the correct SITE_ID. Test with curl directly against 8.208.16.168:9310.
Hourly data chart empty	GET /DailyDataMeter/readHourly requires FROM/TO within the last 7 days for fresh data. Check that date range is not too far in the past.
Codex modifies wrong files	Add explicit guard: 'Do NOT modify any file other than the ones listed below.' at the start of your task prompt.
TypeScript errors after Codex task	Ask Codex: 'Run npx tsc --noEmit and fix all TypeScript errors.' Paste the error output into the follow-up task if needed.
CORS error in browser	Add your frontend URL to CORS_ORIGINS in backend .env. Example: CORS_ORIGINS=http://localhost:5173,https://your-domain.com
Token format wrong (not 20 digits)	The Odyssey API returns tokens in its own format. Do not add hyphens server-side — format as XXXXX-XXXXX-XXXXX-XXXXX only in the frontend display layer.
Promise.allSettled returns all rejected	Check network connectivity to 8.208.16.168:9310 from the server. Run: curl -v http://8.208.16.168:9310/api/health to test connectivity.

11. Quick Reference

Site Colors

Site	Color Hex
KYAKALE	#06D6A0 (emerald green)
MUSHA	#00B4D8 (cyan blue)
UMAISHA	#FFB703 (amber)
TUNGA	#FB8500 (orange)
OGUFA	#EF4444 (red)

Confirmed Working Endpoints (from Postman)

Name	Endpoint
Credit Token Records	GET /api/token/creditTokenRecord/readMore?FROM=&TO=&SITE_ID=
Hourly AMR Data	GET /api/DailyDataMeter/readHourly?offset=0&pageLimit=100&FROM=&TO=&SITE_ID=

Key URLs

Resource	URL
OpenAI Codex	https://codex.openai.com
Odyssey API	http://8.208.16.168:9310/api
Local backend	http://localhost:4000
Local frontend	http://localhost:5173
Backend health	http://localhost:4000/health
JWT decoder	https://jwt.io (verify/inspect token claims)

Environment Variables

Variable	Value
ODYSSEY_API_URL	http://8.208.16.168:9310/api
ODYSSEY_JWT_TOKEN	Your full Bearer JWT — never commit to git
PORT	4000
NODE_ENV	development or production
CORS_ORIGINS	Comma-separated allowed origins
MAX_RETRIES	3 (API retry attempts)
CACHE_TTL_MS	300000 (5-minute dashboard cache)